

PRACTICAL_WEEK-07

Task 1: Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address-space layout and explain each memory segment.

```
UW PICO 5.09

#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

void sample_function()
{
    printf("Inside sample function\n");
}

int main()
{
    static int static_var = 200;
    int stack_var = 300;

    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 400;

    printf("Code / Text Segment Address : %p\n", (void *)sample_function);
    printf("Global Variable Address      : %p\n", (void *)&global_var);
    printf("Static Variable Address      : %p\n", (void *)&static_var);
    printf("Heap Variable Address           : %p\n", (void *)heap_var);
    printf("Stack Variable Address          : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Fig 1: C program to display addresses of code, global, static, heap, and stack memory segments

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/week7_task1.c -o bin/week7_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/week7_task1
Code / Text Segment Address : 0x104dec4f8
Global Variable Address      : 0x104df4000
Static Variable Address      : 0x104df4004
Heap Variable Address        : 0x1054e58d0
Stack Variable Address       : 0x16b013028
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 2: Execution output showing addresses of different process memory segments

```

amrutha@Amruthas-MacBook-Air OSSP % cat week7_task1_flowchart.txt
+-----+
|  START  |
+-----+
|         |
|   v     |
+-----+
| Declare Global Variable |
+-----+
|         |
|   v     |
+-----+
| Define Sample Function |
| for Code Segment      |
+-----+
|         |
|   v     |
+-----+
| Declare Static Variable |
+-----+
|         |
|   v     |
+-----+
| Declare Stack Variable  |
+-----+
|         |
|   v     |
+-----+
| Allocate Heap Memory    |
| using malloc()          |
+-----+
|         |
|   v     |
+-----+
| Print Address of Code   |
| Segment                 |
+-----+
|         |
|   v     |
+-----+
| Print Global & Static   |
| Variable Addresses      |
+-----+
|         |
|   v     |
+-----+
| Print Heap & Stack      |
| Variable Addresses      |
+-----+
|         |
|   v     |
+-----+
| Free Heap Memory        |
+-----+
|         |
|   v     |
+-----+
|  END  |
+-----+
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 3: Flowchart for analyzing code, global, static, heap, and stack memory addresses

Observation:

The program displayed different memory addresses for code, global, static, heap, and stack variables, showing that each type of data is stored in a separate region of the process address space.

Result:

The process address space was analyzed successfully by displaying the addresses of code, global, static, heap, and stack variables. The experiment demonstrated the organization of different memory segments in a running process.

Task 2: Make use of the `/proc/<PID>/maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

int main()
{
    int stack_var = 20;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 30;

    printf("Process PID: %d\n", getpid());
    printf("Global variable address: %p\n", (void *)&global_var);
    printf("Heap variable address: %p\n", (void *)heap_var);
    printf("Stack variable address: %p\n", (void *)&stack_var);

    printf("\nProcess is running for memory analysis...\n");

    sleep(120);

    free(heap_var);

    return 0;
}
```

Fig 4: C program created to keep a process running for virtual memory analysis

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/week7_task2.c -o bin/week7_task2
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/week7_task2 &
[1] 19378
amrutha@Amruthas-MacBook-Air OSSP % Process PID: 19378
Global variable address: 0x1041ac000
Heap variable address: 0x1048a9a90
Stack variable address: 0x16bc5b028

Process is running for memory analysis...
Press Ctrl+C after completing the analysis.
[PID=$!]
[amrutha@Amruthas-MacBook-Air OSSP % echo $PID
19378
[amrutha@Amruthas-MacBook-Air OSSP % ps -p "$PID" -o pid,ppid,state,command
PID  PPID  STAT  COMMAND
19378 19107  SN    ./bin/week7_task2
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 5: Running the memory-analysis program in the background and verifying its process ID

```

[amrutha@Amruthas-MacBook-Air OSSP % vmmap $PID
Process:      week7_task2 [19378]
Path:         /Users/USER/*/week7_task2
Load Address: 0x1041a4000
Identifier:    week7_task2
Version:      0
Code Type:    ARM64
Platform:     macOS
Parent Process: zsh [19107]
Target Type:   live task

Date/Time:     2026-09-19 17:51:17.986 +0538
Launch Time:   2026-09-19 17:49:38.167 +0538
OS Version:    macOS 26.0.2 (25G83)
Report Version: 7
Analysis Tool: /usr/bin/vmmap

Physical footprint: 928K
Physical footprint (peak): 968K
Idle exit:        untracked
=====
Virtual Memory Map of process 19378 (week7_task2)
Output report format: 2.4 -- 64-bit process
VM page size: 16384 bytes
Collected with PhysFootprint mode enabled

===== Non-writable regions for process 19378
REGION TYPE                START - END [ VSIZE RSDNT DIRTY SWAP] PRT/MAX SHRMODE PURGE REGION DETAIL
-----
__TEXT                     1041a4000-1041a8000 [ 16K 16K 0K 0K] r--/r-x SM-COW /Users/amrutha/OSSP/bin/week7_task2
__DATA_CONST              1041a8000-1041ac000 [ 16K 16K 0K 0K] r--/rw SM-COW /Users/amrutha/OSSP/bin/week7_task2
__LINKEDIT                1041b0000-1041bd000 [ 16K 16K 0K 0K] r--/r SM-COW /Users/amrutha/OSSP/bin/week7_task2
Shared memory             1041bc000-1041c000 [ 32K 32K 32K 0K] r--/r SM-SHM
MALLOC guard page         1041c0000-1041c0000 [ 3129K 0K 0K 0K] ---/--- SM-SHM
MALLOC metadata           10477c000-104780000 [ 16K 16K 16K 0K] r--/rw SM-PRV
MALLOC guard page         1048a0000-1048a0000 [ 356K 0K 0K 0K] ---/--- SM-SHM
MALLOC guard page         104970000-104970000 [ 16K 0K 0K 0K] ---/rw SM-SHM
MALLOC guard page         104980000-104980000 [ 16K 0K 0K 0K] ---/rw SM-SHM
STACK GUARD               16765c000-16b460000 [ 56.0M 0K 0K 0K] ---/rw SM-NUL stack guard for thread 0
__TEXT                    185340000-185390000 [ 332K 332K 0K 0K] r-x/r-x SM-COW /usr/lib/libobjc.A.dylib
__TEXT                    185390000-1853d0000 [ 212K 116K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libdyld.dylib
__TEXT                    1853d0000-185480000 [ 728K 728K 0K 0K] r-x/r-x SM-COW /usr/lib/dyld
__TEXT                    185480000-1854d0000 [ 16K 16K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_blocks.dylib
__TEXT                    1854d0000-1854e0000 [ 348K 348K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libcpc.dylib
__TEXT                    1854e0000-1854f0000 [ 132K 132K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_trace.dylib
__TEXT                    1854f0000-1855a0000 [ 768K 684K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libcorecrypto.dylib
__TEXT                    1855a0000-1855f0000 [ 324K 324K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_malloc.dylib
__TEXT                    1855f0000-185640000 [ 288K 288K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libdispatch.dylib
__TEXT                    185640000-185690000 [ 12K 12K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_featureflags.dylib
__TEXT                    185690000-1856c0000 [ 528K 488K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_c.dylib
__TEXT                    1856c0000-185750000 [ 568K 452K 0K 0K] r-x/r-x SM-COW /usr/lib/libc++.1.dylib
__TEXT                    185750000-185770000 [ 188K 68K 0K 0K] r-x/r-x SM-COW /usr/lib/libc++abi.dylib
__TEXT                    185770000-1857d0000 [ 248K 248K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_kernel.dylib
__TEXT                    1857d0000-1857e0000 [ 52K 52K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_pthread.dylib
__TEXT                    1857e0000-1857f0000 [ 36K 36K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_platform.dylib
__TEXT                    1857f0000-1857f0000 [ 192K 192K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_info.dylib
__TEXT                    1857f0000-1857f0000 [ 48K 48K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_darwin.dylib
__TEXT                    185800000-185810000 [ 76K 76K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_notify.dylib
__TEXT                    185810000-1858e0000 [ 188K 92K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_networkextension.dylib
__TEXT                    1858e0000-1858e0000 [ 96K 96K 0K 0K] r-x/r-x SM-COW /usr/lib/system/libsystem_asl.dylib
=====

```

Fig 6: Virtual memory mapping of the running process using the macOS vmmap utility

```

[amrutha@Amruthas-MacBook-Air OSSP % ls /proc
ls: /proc: No such file or directory
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 7: Verification that the Linux /proc filesystem is unavailable on macOS

```

[amrutha@Amruthas-MacBook-Air OSSP % nano week7_task2_flowchart.txt
[amrutha@Amruthas-MacBook-Air OSSP % cat week7_task2_flowchart.txt
+-----+
|           START           |
+-----+
|           |
|           v               |
+-----+
| Create variables in       |
| Global, Heap and Stack   |
+-----+
|           |
|           v               |
+-----+
| Display Process PID      |
| and Memory Addresses     |
+-----+
|           |
|           v               |
+-----+
| Keep Process Running     |
+-----+
|           |
|           v               |
+-----+
| Verify Running Process   |
| using ps                 |
+-----+

```



Fig 8: Flowchart for analyzing the virtual memory organization of a running process

Observation:

The virtual memory layout of the running process was examined using `vmmap`. Different memory regions such as executable code, data, heap, stack, and shared libraries were observed. The Linux `/proc/<PID>/maps` interface was unavailable on macOS.

Result:

The virtual memory organization of a running process was successfully analyzed. The process memory mappings were examined using the macOS `vmmap` utility, and the organization of code, data, heap, stack, and shared-library regions was studied.